A Software Engineering Framework for Reproducible Quantum Algorithm Development: Maturity Gates, Automated Validation, and Honest Limitations Across 20 Problem Domains

Authors

Werner Rall

Abstract

We present a software engineering framework for organizing quantum algorithm development across multiple problem domains. The framework provides standardized project structure, automated CI/CD validation, and a four-stage maturity gate model that explicitly prevents premature quantum advantage claims. We apply the framework to 20 problem domains using Microsoft Q#, implementing nine algorithm families at toy scale (2-8 qubits) where classical simulation is trivial. **We do not claim quantum advantage for any problem.** All implementations use small, highly structured instances specifically chosen for correctness validation, not for demonstrating quantum utility. Simulator results confirm algorithmic correctness but reveal nothing about noise resilience or practical scalability. The primary contribution is the methodology itself — particularly the maturity gate model that forces honest assessment of what has and has not been demonstrated — not the individual quantum implementations. We include a scaling analysis for Grover search showing the expected $O(\sqrt{N})$ query complexity alongside the corresponding growth in circuit depth and gate count that would be required for practically relevant problem sizes. The framework and all implementations are open-source (DOI: 10.5281/zenodo.19222021).

Keywords: quantum software engineering, development methodology, maturity gates, reproducibility, Q#, CI/CD

1. Introduction

The quantum computing community faces a credibility challenge. Claims of quantum advantage have repeatedly been retracted when classical heuristics proved more competitive than initially assumed [5], when constant-factor costs of quantum error correction were properly accounted [6], or when problem structure was exploited by classical algorithms post-hoc. A contributing factor is the absence of standardized development practices that enforce honest assessment at each stage of quantum algorithm development.

This paper addresses not the algorithms themselves — which are well-established in the literature — but the **software engineering process** of developing, validating, and honestly assessing quantum implementations. We propose a framework that applies the same rigor to quantum development that software engineering applies to classical systems: standardized structure, automated testing, evidence-based quality gates, and reproducible benchmarks.

Scope and limitations. Our 20 implementations are toy-scale (2-8 qubits) where classical simulation is trivial. We use standard textbook algorithms applied to small structured instances. We do not advance quantum algorithm theory, discover new speedups, or demonstrate quantum utility. The contribution is purely methodological: a framework that prevents the common failure mode of claiming more than the evidence supports.

1.1 Contributions

1. **Maturity Gate Model**: A four-stage procedural framework (A→D) with explicit evidence requirements at each stage, designed to prevent premature quantum advantage claims. This is a process contribution, not a complexity-theoretic one.
2. **Standardized Problem Structure**: A reproducible template ensuring every problem includes classical baselines, parameterized instances, quantum implementations, and honest limitations documentation.
3. **Automated Policy Enforcement**: CI/CD pipelines that reject merges when maturity claims are not backed by evidence artifacts.
4. **Scaling Analysis**: For one representative problem (Grover search), we show how circuit depth, gate count, and resource requirements grow with problem size, illustrating the gap between toy validation instances and practically relevant scales.
5. **Lessons Learned**: Practical insights from building 20 quantum implementations, including how compiled-but-incorrect code, encoding bugs, and mock estimates can create false confidence.

2. Related Work

Prior benchmarking efforts include the QED-C benchmark suite [1], which focuses on circuit-level metrics, and the QUARK framework [2] for optimization problems. IBM's Qiskit benchmarks [3]

provide hardware-focused comparisons. Our work differs in scope (20 domains vs. single-domain), methodology (maturity gates vs. raw performance), and emphasis on reproducibility infrastructure.

3. Framework Design

3.1 Problem Structure

Each of the 20 problems follows an identical directory layout:

```
problems/XX_name/
├── qsharp/          # Q# quantum implementation
├── python/          # Classical baseline and analysis
├── instances/       # Parameterized YAML configurations (small/medium/large)
├── estimates/       # Resource estimation artifacts
├── plots/           # Generated visualizations
├── Makefile         # Reproducible build commands
└── README.md        # Documentation with advantage claim contract
```

This uniformity enables automated tooling to operate across all 20 problems without problem-specific logic.

3.2 Maturity Gate Model

We define four progressive stages, each with explicit evidence requirements:

Stage A (Classical Baseline Validated): - Deterministic classical baseline implementation - Instance coverage (small, medium at minimum) - Reproducible command path documented - Metrics persisted in standardized JSON schema

Stage B (Quantum Implementation Ready): - Q# quantum algorithm implemented with real gate operations - Build and run validation passing in CI - Resource estimation path documented - Algorithm assumptions and known limits stated

Stage C (Hardware-Aware Validation): - Multi-run calibration with uncertainty bounds - Quantum/classical comparison with confidence intervals - Hardware mapping and transpilation assumptions documented - DiVincenzo readiness criteria assessed

Stage D (Advantage Evidence Package): - Advantage claim template completed with explicit category (theoretical/projected/demonstrated) - Fair classical comparator identified and justified - Residual risks and assumption sensitivities documented - Reproducible end-to-end evidence chain

3.3 Advantage Claim Contract

Each problem includes a structured advantage claim with fields for: claim category, problem class and regime, baseline algorithm and fairness rationale, quantum resource scaling claim, data-loading and I/O assumptions, noise/error model assumptions, confidence/uncertainty method, and residual risks. This prevents implicit or unsupported advantage claims.

3.4 DiVincenzo Readiness Overlay

For Stage C/D problems, we assess five DiVincenzo criteria: scalable qubit system, initialization capability, coherence relative to gate times, universal gate set, and qubit-specific measurement. Each criterion is tagged as met, partial, not-yet, or not-applicable with evidence notes.

4. Implementation

4.1 Toolchain

- **Quantum:** Microsoft Q# with modern QDK (qsharp 1.27, Python-hosted, no .NET dependency)
- **Classical:** Python 3.11 with NumPy, SciPy, Matplotlib
- **Cloud:** Azure Quantum with Quantinuum and Rigetti providers
- **CI/CD:** GitHub Actions with 7 automated checks
- **Website:** Next.js dashboard with live KPI visualization

4.2 Algorithm Families

Table 1 summarizes the 9 algorithm families implemented across 20 problems.

Algorithm Family	Problems	Qubits	Key Operations
VQE	01, 02, 07, 14, 17	2	Ry, CNOT, Rz, Pauli measurement
QAOA	05, 08, 12, 20	3-4	H, ZZ cost (CNOT+Rz), Rx mixer
Grover	10, 15	3-5	Oracle marking, diffusion operator
HHL/QPE	04, 13	5-6	QPE, QFT, eigenvalue inversion
IQAE	03, 06	5	Iterative: state prep, oracle, Grover^k, measure — no QPE register
Shor	09	8	QPE, controlled modular multiply

Algorithm Family	Problems	Qubits	Key Operations
Swap Test	11	5	Controlled-SWAP, Hadamard test
QEC	16	5	Syndrome extraction, correction
Quantum Walk	18	3	Coin rotation, conditional shift
Trotter	19	4	ZZ plaquettes, transverse field

4.3 Classical Baselines

Every problem includes a deterministic classical baseline that serves as the comparison target. Baseline algorithms are chosen to be the standard classical approach for each domain (e.g., exact diagonalization for Hubbard, Monte Carlo for risk analysis, brute-force for MaxCut at small scale). All baselines produce JSON output conforming to a shared schema.

4.4 Resource Estimation Pipeline

We employ a centralized estimation manager (`tooling/generate_estimates.py`) that runs `qsharp.estimate()` on each problem's kernel operation. All 20 problems now have real Azure Quantum Resource Estimator profiles with surface_code architecture targeting. Physical qubit counts range from 1,764 (3-qubit QEC repetition code) to 401,400 (VQE band gap with nested optimization). The pipeline also supports batch calibration via `tooling/generate_calibration_ensemble.py` , which runs each kernel 20 times to produce ensemble statistics with 95% confidence intervals.

Current target architectures: - `surface_code_generic_v1` : Generic surface code with standard parameters - `qubit_gate_ns_e3` : Gate-based model, 1µs gate time, 10^{-3} error rate - `qubit_gate_ns_e4` : Gate-based model, 1µs gate time, 10^{-4} error rate

5. Automated Validation

5.1 CI Pipeline

Seven automated checks run on every pull request:

1. **Build and Test**: Compiles all 20 Q# projects, runs Python baselines
2. **Stage D Readiness Gate**: Validates maturity stage claims against evidence
3. **Runnable Correctness Gate**: Executes end-to-end correctness checks
4. **Reporting Integrity Gate**: Ensures KPI artifacts are fresh and schema-compliant

5. **Azure Secret Hygiene**: Prevents accidental credential commits
6. **Quick Checks**: Fast lint and schema validation
7. **Security Scanning**: Automated secret detection

5.2 KPI Dashboard

An automated reporting script (`stage_kpis.py`) scans all 20 problem directories and produces a maturity status report with four flags per problem: advantage contract present, estimator profile summary generated, backend assumptions documented, and DiVincenzo readiness assessed. The CI pipeline enforces that all required flags are satisfied before merging.

Current coverage: 20/20 problems have advantage contracts, estimator summaries, and backend assumptions.

6. Azure Quantum Integration

6.1 Circuit Validation

Twenty quantum circuits have been validated on the Quantinuum H2-1SC trapped-ion syntax checker via Azure Quantum, compiled from Q# to QIR using the modern QDK (qsharp 1.27):

Circuit	Algorithm	Qubits	Gates	Status
Hubbard VQE (ZZ basis)	VQE	2	~8	Succeeded
Hubbard VQE (XX basis)	VQE	2	~10	Succeeded
MaxCut QAOA depth-1	QAOA	3	~12	Succeeded
Database Grover 4-qubit	Grover	4	~200	Succeeded
Key Search Grover	Grover	4	~150	Succeeded
Scheduling QAOA	QAOA	4	~24	Succeeded
QEC Repetition Code	QEC	5	~10	Succeeded
Shor N=15 (a=7)	Shor	8	~50	Succeeded
HHL Diffusion PDE	HHL	5	~30	Succeeded
QAE Simplified	QAE	4	~20	Succeeded
Quantum VaR	QAE	3	~10	Succeeded

Circuit	Algorithm	Qubits	Gates	Status
QAOA Protein Folding	QAOA	4	~30	Succeeded
VQE Catalysis	VQE	2	~6	Succeeded
VQE Drug Binding	VQE	2	~8	Succeeded
Swap Test Kernel	Swap	5	~10	Succeeded
VQE Band Gap	VQE	2	~6	Succeeded
VQE Deuteron	VQE	2	~8	Succeeded
Quantum Walk Exciton	Walk	3	~8	Succeeded
Trotter Gauge	Trotter	4	~40	Succeeded
QAOA Trajectory	QAOA	4	~30	Succeeded

6.2 Shared Azure Workflow

A problem-agnostic Azure submission pipeline (`tooling/azure/smoke_problem.py`) enables any problem to submit circuits to Azure Quantum through a single interface, with env-gated authentication, manifest tracking, and audit reporting.

7. Results and Discussion

7.1 What the Framework Demonstrates (and What It Does Not)

The maturity gate model successfully enforces honest assessment: of 20 implemented problems, all 20 have reached Stage C (hardware-aware validation with 20-run calibration ensembles and real Azure Quantum Resource Estimator profiles), and **4 have reached Stage D** (advantage evidence with explicit claim contracts). The 4 Stage D candidates (QAE, QAOA MaxCut, Grover DB Search, Grover PQC) each filed honest advantage claim contracts: QAE and QAOA as "theoretical" (no practical advantage demonstrated), Grover variants as "projected" (provably optimal $O(\sqrt{N})$ speedup, practical at $N>10^6$).

Critically, the framework prevented inflated claims: the QAOA MaxCut Stage D contract explicitly states "no proven constant-depth advantage" and identifies the Goemans-Williamson 0.878-approximation as the polynomial-time classical competitor.

What our results demonstrate: - Algorithmic correctness at small scale (Grover finds marked items, Shor factors 15, QEC corrects single bit-flips) - The framework's ability to organize and validate

quantum development across multiple problem domains - Practical lessons about failure modes in quantum software development

What our results do NOT demonstrate: - Quantum advantage over classical methods for any problem - Noise resilience or hardware viability - Scalability to practically relevant problem sizes - Superiority over state-of-the-art classical algorithms

7.2 Simulator Results Are Sanity Checks, Not Experiments

All validation was performed on noiseless simulators or the Quantinuum H2-1SC syntax checker. Zero-variance results (e.g., 100% QEC correction rate, exact QAOA optimality) confirm correct circuit construction but reveal nothing about performance under realistic noise. A circuit that achieves perfect results on a simulator may fail completely on hardware with 10^{-3} two-qubit gate error rates.

The H2-1SC syntax checker validates that circuits are well-formed QIR (compiled from Q# via the modern QDK) but does not simulate quantum behavior. Full emulator (H2-1E) results were collected for all 20 problems (100 shots each), with cross-platform validation on 19 problems via Rigetti QVM. Key finding: **17/19 problems agree on the dominant measurement outcome across both platforms**, providing strong cross-platform consistency evidence. A separate depolarizing noise simulation study at three error rates (p=0.001, 0.01, 0.05) revealed that 2-qubit VQE circuits maintain >90% fidelity even at p=0.05, while deeper circuits (Grover with 3 iterations, 8-qubit Shor) degrade to <40% fidelity — quantifying the noise sensitivity gap that separates near-term and fault-tolerant algorithm classes.

Selected correctness validation results: - **Grover (10_pqc)**: 80-92% success rate across 3-5 qubit instances on noiseless simulator - **Shor (09_factorization)**: Correctly factors 15 = 3 × 5 (a textbook demonstration, not a research contribution) - **QEC (16_error_correction)**: 512/512 = 100% correction on simulator (meaningless without noise — the entire point of QEC is noise resilience)

7.3 Scaling Analysis: Grover Search (Computed)

To illustrate the gap between toy validation and practical utility, we computed Grover search resource requirements at increasing qubit counts, using standard Toffoli-based decomposition of multi-controlled Z gates (each Toffoli = 6 CX + 7 T gates).

Qubits (n)	Keyspace (N)	Iterations	CX Gates	T Gates	Total Gates	Success (noiseless)
3	8	2	24	28	84	94.5%
4	16	3	72	84	222	96.1%
5	32	4	144	168	424	99.9%

Qubits (n)	Keyspace (N)	Iterations	CX Gates	T Gates	Total Gates	Success (noiseless)
6	64	6	288	336	828	99.7%
8	256	12	864	1,008	2,424	100%
10	1,024	25	2,400	2,800	6,650	100%
12	4,096	50	6,000	7,000	16,500	100%
16	65,536	201	33,768	39,396	92,058	100%
20	1,048,576	804	173,664	202,608	471,144	100%

The quadratic speedup ($O(\sqrt{N})$ queries) is real in query complexity, but total gate count grows as $O(\sqrt{N} \times n)$ where n is the number of qubits. For n=20 ($N \approx 10^6$), the circuit requires ~471K gates. For cryptographically relevant keyspaces (n=128 for AES), the circuit would require approximately 10^{19} gates, far beyond any foreseeable hardware. **No amount of CI/CD rigor changes these scaling realities.**

7.3.1 Noise Impact on Grover Scaling

Using a depolarizing noise model where each two-qubit gate has error probability 10× the single-qubit rate, we computed the impact on Grover success probability:

Qubits	2Q Gates	Noiseless	p=0.1%	p=1%	p=5%
3	24	94.5%	76.0%	17.2%	12.5%
4	72	96.1%	49.5%	6.3%	6.2%
5	144	99.9%	24.0%	3.1%	3.1%
6	288	99.7%	5.7%	1.6%	1.6%
8	864	100%	0.4%	0.4%	0.4%

At a realistic two-qubit gate error rate of 1%, Grover success collapses from 96% to 6% at 4 qubits and becomes indistinguishable from random guessing at 6+ qubits. This demonstrates that noiseless simulator results (Section 7.2) reveal nothing about practical viability. Error correction would be required for any non-trivial Grover instance, adding orders of magnitude in qubit overhead.

7.4 Classical Baseline Honesty: Goemans-Williamson vs QAOA

To illustrate why our classical baselines are insufficient for advantage claims, we compared QAOA MaxCut against the Goemans-Williamson (GW) SDP relaxation [7]:

Graph	Nodes	Optimal Cut	GW Guarantee (≥0.878×OPT)	Brute-Force Time	GW Time
Triangle	3	2.2	≥1.93	$O(2^3) = 8$	$O(3^3) = 27$
Square+diag	4	4.0	≥3.51	$O(2^4) = 16$	$O(4^3) = 64$
Pentagon+diags	5	5.4	≥4.74	$O(2^5) = 32$	$O(5^3) = 125$

At our problem scales ($n \leq 5$), brute-force enumeration, GW-SDP, and QAOA all find the optimal cut trivially. The GW 0.878-approximation guarantee only becomes relevant when brute-force is infeasible ($n > 30$). **Claiming QAOA advantage against brute-force at n=3 is meaningless** — this is precisely the kind of inflated claim that our maturity gate framework is designed to prevent.

7.5 Lessons Learned

1. **Compiled code ≠ correct quantum code**: Analytical Q# code that runs classical math with a token qubit allocation compiles, passes CI, and produces plausible output — but performs no quantum computation. Fifteen of our twenty implementations were initially such placeholders. Gate-count auditing is essential.

2. **Encoding bugs persist through validation**: A Lorentzian PDF masquerading as a log-normal distribution in the QAE implementation survived multiple development cycles because the circuit compiled and produced numbers in a plausible range. Only systematic comparison against the classical baseline revealed the error.

3. **Mock estimates previously created false confidence**: Uniform mock resource estimates across all problems suggested comparable resource requirements. Real Azure Quantum Resource Estimator profiles now reveal a 230× range: from 1.8k physical qubits (QEC repetition code) to 401k (VQE band gap with nested optimization). T-gate intensive algorithms (QAE: 15, HHL: 12, Shor: 6) require T-state distillation factories, while VQE/QAOA problems use only rotation gates.

4. **Process discipline has value even without quantum advantage**: The standardized structure, automated checks, and maturity gates caught real bugs and prevented inflated claims — which is the framework's actual contribution.

8. Limitations (Critical)

These limitations are fundamental to interpreting this work, not merely areas for improvement:

1. **No quantum advantage is claimed or demonstrated.** All problems are solved trivially by classical computers. The quantum implementations exist to validate the framework methodology, not to demonstrate quantum utility.

2. **All results are from noiseless simulation.** Zero-variance results (100% QEC correction, exact QAOA optimality) are artifacts of simulation, not evidence of hardware viability. Real quantum hardware with 10^{-3} to 10^{-2} error rates would produce dramatically different results.

3. **Classical baselines are deliberately weak.** We use textbook algorithms (brute-force, naive Monte Carlo) for cross-domain standardization. Any comparative statement must acknowledge that state-of-the-art classical methods would perform far better. Quantum advantage claims have repeatedly vanished when classical heuristics were properly benchmarked [6].

4. **Resource estimates now use real Azure Quantum Resource Estimator profiles.** All 20 problems have been profiled via `qsharp.estimate()` with the surface_code architecture target. Physical qubit counts range from 1,764 (QEC) to 401,400 (Materials Discovery). These are credible lower bounds for fault-tolerant execution, though they apply to toy-scale instances and should not be extrapolated to production problem sizes without careful scaling analysis.

5. **Toy instances reveal nothing about asymptotic behavior.** A 2-qubit VQE or 4-qubit QAOA demonstrates circuit correctness but says nothing about how the algorithm behaves at 50, 100, or 1000 qubits. The scaling analysis in Section 7.3 illustrates how rapidly resource requirements grow.

6. **The maturity gate model is procedural, not complexity-theoretic.** It enforces process discipline but cannot compensate for unfavorable asymptotic scaling. A problem that passes all four maturity gates may still offer no practical quantum advantage if the constant factors or scaling exponents are prohibitive.

9. Future Work

- ~~Obtain real Azure Quantum Resource Estimator profiles~~ (completed April 2026: all 20 problems profiled)
- Execute 2-qubit circuits on Quantinuum H1 QPU to compare noisy hardware results against simulator baselines
- Replace at least one classical baseline with a state-of-the-art competitor (Goemans-Williamson for MaxCut, importance sampling for QAE)
- Extend scaling analysis to QAOA and VQE with noise models at different error rates
- Investigate whether the maturity gate model can be extended with complexity-theoretic checks (scaling slope verification, classical hardness evidence requirements)

- Promote Stage D candidates (QAE, QAOA, Grover) with full advantage evidence packages

10. Conclusion

We have presented a software engineering framework for quantum algorithm development that prioritizes honest assessment over optimistic claims. The framework's primary value is not in the quantum implementations — which are toy-scale and classically trivial — but in the methodology: standardized structure, automated validation, and maturity gates that explicitly prevent claiming more than the evidence supports.

Of 20 implemented problems, all have reached Stage C (hardware-aware validation with calibration ensembles and real resource estimates), and 4 have reached Stage D (advantage evidence with filed claim contracts). No problem claims demonstrated practical quantum advantage — the Stage D contracts are explicitly tagged as "theoretical" or "projected" with documented residual risks. The maturity gate model correctly identifies that emulator results on small instances, even with cross-platform validation, do not constitute evidence of quantum utility.

The practical lessons — that placeholders masquerade as implementations, that encoding bugs survive validation, that mock estimates create false confidence — are transferable to any quantum development effort. We hope the framework's insistence on honest self-assessment contributes to healthier practices in the quantum computing community.

All code, data, and tooling are available at https://github.com/WernerRall147/quantum-grand-challenges (DOI: 10.5281/zenodo.19222021).

References

[1] T. Lubinski et al., "Application-Oriented Performance Benchmarks for Quantum Computing," IEEE Transactions on Quantum Engineering, 2023.

[2] J. Finžgar et al., "QUARK: A Framework for Quantum Computing Application Benchmarking," IEEE International Conference on Quantum Computing and Engineering, 2022.

[3] A. Javadi-Abhari et al., "Quantum Computing with Qiskit," arXiv:2405.08810, 2024.

[4] M. A. Nielsen and I. L. Chuang, "Quantum Computation and Quantum Information," Cambridge University Press, 2010.

[5] J. Preskill, "Quantum Computing in the NISQ era and beyond," Quantum, 2018.

[6] R. Babbush et al., "Focus beyond quadratic speedups for error-corrected quantum advantage," PRX Quantum, 2021.

[7] M. Goemans and D. Williamson, "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming," JACM, 1995.

Appendix A: Problem Registry

#	Domain	Algorithm	Qubits	Stage
01	Hubbard Model	VQE	2	B
02	Catalysis	VQE	2	B
03	Risk Analysis	IQAE	5	C
04	Linear Solvers	HHL	6	B
05	MaxCut	QAOA	3	C
06	Trading	QAE	3	B
07	Drug Discovery	VQE	2	B
08	Protein Folding	QAOA	4	B
09	Factorization	Shor	8	B
10	Cryptography	Grover	3-5	B
11	Machine Learning	Swap Test	5	B
12	Optimization	QAOA	4	B
13	Climate	HHL	6	B
14	Materials	VQE	2	B
15	Database Search	Grover	4-12	C
16	Error Correction	QEC	5	B
17	Nuclear Physics	VQE	2	B
18	Photovoltaics	Quantum Walk	3	B
19	QCD	Trotter	4	B
20	Space Planning	QAOA	4	B

Generated: April 2026 | License: CC BY-NC-SA 4.0 | Repository: github.com/WernerRall147/quantum-grand-challenges